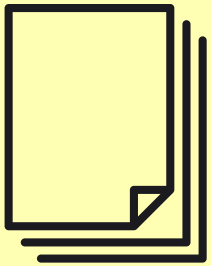
Starter: Active recalled



On your own:

Make a *brain dump* on the topic of the **call stack**.

Aim to include:

- Where is the call stack?
- What information is stored in the call stack?
- How does it change in size during runtime?

Stacks

Keywords: Stack; Data structure; Abstract Data type; Push; Pop; Peek; Stack overflow;

```
try {
    // Paste entire codebase here
} catch (Exception ex) {
    var q = System.Web.HttpUtility.UrlEncode(ex.Message);
    Process.Start("https://stackoverflow.com/search?q=" + q);
}
```

A ‘Stack Overflow’ Exception

Stacks

Keywords: Stack; Data structure; Abstract Data type; Push; Pop; Peek; Stack overflow;

Stacks

4.2.1.4 Abstract data types/data structures

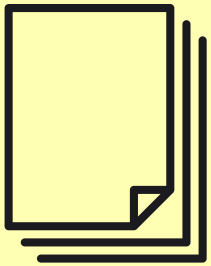
Content	Additional information
Be familiar with the concept and uses of a stack	Be able to use this abstract data type and its equivalent data structures in simple contexts. Students should also be familiar with methods for representing it when a programming language does not support the structure as a built-in type.
Describe the creation and maintenance of data within stacks.	

4.2.3.1 Stacks

Content	Additional information
Be able to describe and apply the following operations: • push • pop • peek or top • test for empty stack • test for stack full.	Peek or top returns the value of the top element without removing it.

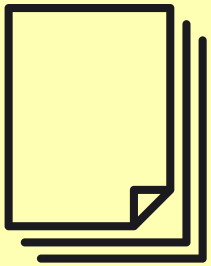
Keywords: Stack; Data structure; Abstract Data type; Push; Pop; Peek; Stack overflow;

Abstract Data Types



We're going to be using **abstract data types** throughout this course.

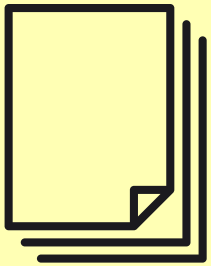
Abstract Data Types



We're going to be using **abstract data types** throughout this course.

They are a theoretical way of organising a collection of data, **independent of any programming language**.

Abstract Data Types

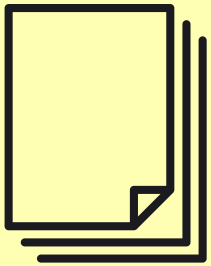


We're going to be using **abstract data types** throughout this course.

They are a theoretical way of organising a collection of data, **independent of any programming language**.

They can then be implemented within a programming language to create a **Data Structure**.

Abstract Data Types



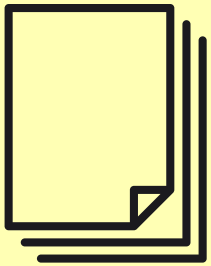
We're going to be using **abstract data types** throughout this course.

They are a theoretical way of organising a collection of data, **independent of any programming language**.

They can then be implemented within a programming language to create a **Data Structure**.

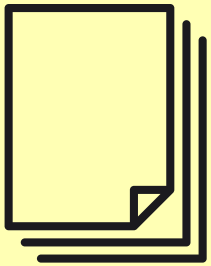
EG: Stacks, Queues, Graphs, Trees, Hash Tables, Dictionaries, Vectors

Abstract Data Types



An **ADT** will carry a list of features or requirements for how we handle the data.

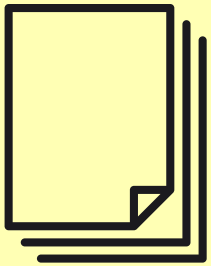
Abstract Data Types



An **ADT** will carry a list of features or requirements for how we handle the data.

For a **stack**, we can consider it a collection of items that can only be accessed from the **top**.

Abstract Data Types

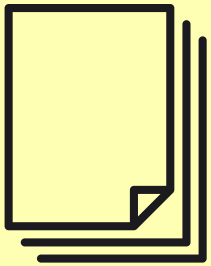


An **ADT** will carry a list of features or requirements for how we handle the data.

For a **stack**, we can consider it a collection of items that can only be accessed from the **top**.

You can put a new item onto the **top**, take an item off of the **top**, or just look at the item on the **top**.

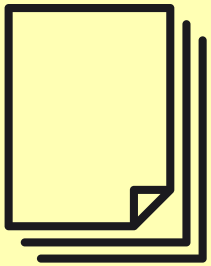
Stack as an ADT



In **abstract** terms, a stack needs to have 5 core operations:

1. `Peek` - return the item at the top without removing it.

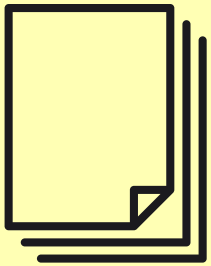
Stack as an ADT



In **abstract** terms, a stack needs to have 5 core operations:

1. `Peek` - return the item at the top without removing it.
2. `Pop` - return the item at the top and remove it.

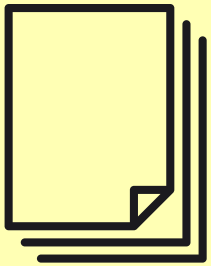
Stack as an ADT



In **abstract** terms, a stack needs to have 5 core operations:

1. `Peek` - return the item at the top without removing it.
2. `Pop` - return the item at the top and remove it.
3. `Push` - Add a new item to the top.

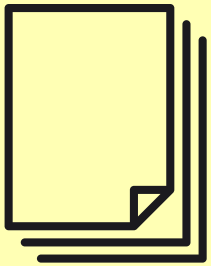
Stack as an ADT



In **abstract** terms, a stack needs to have 5 core operations:

1. `Peek` - return the item at the top without removing it.
2. `Pop` - return the item at the top and remove it.
3. `Push` - Add a new item to the top.
4. `IsEmpty`

Stack as an ADT



In **abstract** terms, a stack needs to have 5 core operations:

1. `Peek` - return the item at the top without removing it.
2. `Pop` - return the item at the top and remove it.
3. `Push` - Add a new item to the top.
4. `IsEmpty`
5. `IsFull`

Dry Run (together)

```
stack.Push(5);
stack.Push(10);
stack.Push(9);

int myInt = stack.Pop();

Console.WriteLine(myInt + stack.Peek());
```

Dry Runs

```
stack.Push(3);
stack.Push(1);
stack.Push(4);
stack.Push(stack.Pop() * stack.Pop();

stack.Push(5);

Console.WriteLine(stack.Pop() + stack.Peek());
```

What does this code output?

What does the stack look like at the end?

Dry Runs

```
stack.Push(3);
stack.Push(1);
stack.Push(4);
stack.Push(stack.Pop() * stack.Pop();

stack.Push(5);

Console.WriteLine(stack.Pop() + stack.Peek());
```

9

4

3

What does this code output?

What does the stack look like at the end?

Dry Runs

```
stack.Push("ll");
stack.Push("o");
stack.Push("He");

string s = stack.Pop();
stack.Push(stack.Pop() + stack.Pop());
stack.Push(s + stack.Pop());
```

What does the stack look like at the end?

Dry Runs

```
stack.Push("ll");
stack.Push("o");
stack.Push("He");

string s = stack.Pop();
stack.Push(stack.Pop() + stack.Pop());
stack.Push(s + stack.Pop());
```

Heoll

What does the stack look like at the end?

Dry Runs

```
stack.Push("lo");
stack.Push("l");
stack.Push("He");

string s = stack.Peek();
stack.Push(stack.Peek() + stack.Peek());
stack.Push(s + stack.Peek());
```

What does the stack look like at the end?

Dry Runs

```
stack.Push("lo");
stack.Push("l");
stack.Push("He");

string s = stack.Peek();
stack.Push(stack.Peek() + stack.Peek());
stack.Push(s + stack.Peek());
```

What does the stack look like at the end?

HeHeHe

HeHe

He

l

lo

Programming an integer stack

1. What are the 5 methods needed for the **stack ADT**?
2. For each of these, what parameters/return types are needed?
3. Write down a **function stub** for each of them.

Static Stacks

A **stack** data structure can be implemented using an array and an integer variable behind the scenes.

Static Stacks

A **stack** data structure can be implemented using an array and an integer variable behind the scenes.

EG:

```
int[] data = new int[5];
int top_pointer = -1;
```

Static Stacks

A **stack** data structure can be implemented using an array and an integer variable behind the scenes.

EG:

```
int[] data = new int[5];
int top_pointer = -1;
```

Thus `Push` would be:

```
top_pointer += 1;
data[top_pointer] = new_item;
```

Discuss

How does the Stack data structure relate to OOP?

Discuss

How does the Stack data structure relate to OOP?

Encapsulation:

1. Combining data and methods into a single object.
2. Restricting access to methods/data.

To create a Stack data structure, we want to do both!

1. Put the data and 5 methods in one place.
2. Restrict access to the data, so you can only access it via the methods.

Thus we could make a Stack class to implement the data structure!

Your turn:

You're going to program an **integer stack** using an array and a top pointer. Select your difficulty:

1. Use the skeleton program provided for the `MyStack` class which has 2 variables and 5 methods required (and a constructor).
2. Write a class called MyStack, which contains the required `private` variables and 5 `public` methods (and a constructor).

Extension: Create a 'generic' `Stack` class rather than an integer-specific one, using *parametric polymorphism*. Research will be required!